

Understanding Python Context Managers

Solutions to Context Manager Exercises

Exercise 1 Solution: Logging Context Protocol

Task: Create a class-based context manager that logs block entry and exit.

```
● ● ● exercise_1_protocol.py

class LoggingContext:
    def __init__(self, name):
        self.name = name

    def __enter__(self):
        print(f"[LoggingContext] Entering block: {self.name}")
        return self

    def __exit__(self, exc_type, exc_val, exc_tb):
        if exc_type is not None:
            print(f"[LoggingContext] Exiting block: {self.name} with exception {exc_type.__name__}")
        else:
            print(f"[LoggingContext] Exiting block: {self.name} successfully")
        return False # Propagate exceptions normally
```

Explanation: The class-based implementation relies on dunder methods. `__enter__` executes at block entry, printing our start log. `__exit__` executes upon block exit, receiving three arguments describing exception states. Returning `False` ensures that errors are raised natively in caller spaces.

Exercise 2 Solution: Safe File Manager Class

Task: Build a robust, class-based file manager that guarantees file closure on exit.

```
● ● ● exercise_2_file_manager.py

class SafeFileManager:
    def __init__(self, filepath, mode):
        self.filepath = filepath
        self.mode = mode
        self.file = None

    def __enter__(self):
        self.file = open(self.filepath, self.mode)
        return self.file

    def __exit__(self, exc_type, exc_val, exc_tb):
        if self.file:
            self.file.close()
        return False
```

Explanation: By deferring file handle initialization to `__enter__` and guaranteeing `self.file.close()` inside `__exit__`, we ensure that file resources are never leaked from memory, regardless of runtime block success.

Exercise 3 Solution: Exception-Aware SQLite Transaction Manager

Task: Build a class-based transaction manager that commits on success or rolls back on exception.

```
● ● ● exercise_3_transaction.py

import sqlite3

class SafeTransaction:
    def __init__(self, db_path):
        self.db_path = db_path
        self.conn = None

    def __enter__(self):
        self.conn = sqlite3.connect(self.db_path)
        return self.conn

    def __exit__(self, exc_type, exc_val, exc_tb):
        if self.conn:
            if exc_type is None:
                self.conn.commit()
                print("[SafeTransaction] Database updates committed successfully.")
            else:
                self.conn.rollback()
                print(f"[SafeTransaction] Exception caught ({exc_type.__name__}): database rolled back.")
                self.conn.close()
        return False
```

Explanation: This transaction manager utilizes the `exc_type` argument. When `exc_type` is `None`, operations were successful and `conn.commit()` is triggered. When `exc_type` holds an exception class, `conn.rollback()` is triggered, ensuring transactional atomicity.

Exercise 4 Solution: Reusable Block Timer Profiler

Task: Build a class-based Timer context manager that measures block execution durations.

```
● ● ● exercise_4_timer.py

import time

class BlockTimer:
    def __init__(self, label="Block"):
        self.label = label
        self.start = None
        self.elapsed = None

    def __enter__(self):
        self.start = time.perf_counter()
        return self

    def __exit__(self, exc_type, exc_val, exc_tb):
        self.elapsed = time.perf_counter() - self.start
        print(f"[{self.label}] Elapsed: {self.elapsed:.6f} seconds")
        return False
```

Explanation: Measuring time intervals is achieved by storing perf_counter values as instance states. By returning self in __enter__, we bind the Timer instance to the with statement target for external post-exit audits.

Exercise 5 Solution: Generator-Based FileManager

Task: Rebuild Exercise 2's SafeFileManager utilizing @contextmanager.

```
● ● ● exercise_5_generator_manager.py

from contextlib import contextmanager

@contextmanager
def safe_file_manager(filepath, mode):
    f = open(filepath, mode)
    try:
        yield f
    finally:
        f.close()
```

Explanation: The @contextmanager decorator hides standard class protocols. Wrapping the yield statement in a try/finally block is mandatory: it guarantees file descriptor closure even when the block raises exceptions.

Exercise 6 Solution: Isolated Temporary Workspace

Task: Create a generator context manager that yields an isolated Path directory and cleans up on exit.

```
● ● ● exercise_6_temp_workspace.py

import os
import shutil
from pathlib import Path
from tempfile import TemporaryDirectory
from contextlib import contextmanager

@contextmanager
def temporary_workspace():
    with TemporaryDirectory() as tmpdir:
        yield Path(tmpdir)
```

Explanation: Utilizing `tempfile`'s `TemporaryDirectory` provides cross-platform sandboxed folders. By nesting `TemporaryDirectory` inside our generator manager, the files and folder are unlinked from disk upon exit.

Exercise 7 Solution: Dynamic Multiple File Handles with ExitStack

Task: Write a function that safely writes headers to a dynamic list of files using ExitStack.

```
● ● ● exercise_7_exit_stack.py

from contextlib import ExitStack

def write_headers_to_all(filepaths, header_text):
    with ExitStack() as stack:
        files = [stack.enter_context(open(path, "w")) for path in filepaths]
        for f in files:
            f.write(header_text + "\n")
```

Explanation: ExitStack behaves as a dynamic manager array. Standard context manager definitions cannot handle dynamic lists natively, but ExitStack registers open descriptors and guarantees cleanup on failure.

Exercise 8 Solution: Capstone Data Processing Pipeline

Task: Implement a robust, timed ETL pipeline utilizing temporary workspaces and transactions.

● ● ● *exercise_8_capstone_pipeline.py*

```
import sqlite3
import csv
import time
from pathlib import Path
from contextlib import contextmanager

class BlockTimer:
    def __init__(self, label="Block"):
        self.label = label
        self.start = None
        self.elapsed = None

    def __enter__(self):
        self.start = time.perf_counter()
        return self

    def __exit__(self, exc_type, exc_val, exc_tb):
        self.elapsed = time.perf_counter() - self.start
        print(f"[{self.label}] Process duration: {self.elapsed:.6f} seconds")
        return False

class SafeTransaction:
    def __init__(self, db_path):
        self.db_path = db_path
        self.conn = None

    def __enter__(self):
        self.conn = sqlite3.connect(self.db_path)
        return self.conn

    def __exit__(self, exc_type, exc_val, exc_tb):
        if self.conn:
            if exc_type is None:
                self.conn.commit()
            else:
                self.conn.rollback()
            self.conn.close()
        return False

@contextmanager
def temporary_workspace():
    from tempfile import TemporaryDirectory
    with TemporaryDirectory() as tmpdir:
        yield Path(tmpdir)

def run_data_pipeline(input_records, db_path):
    with BlockTimer("Pipeline ETL Execution") as timer:
        with temporary_workspace() as stage_dir:
            csv_path = stage_dir / "records.csv"
            with open(csv_path, "w", newline="") as f:
                writer = csv.writer(f)
                writer.writerow(["sku", "name", "quantity"])
                writer.writerows(input_records)
            with SafeTransaction(db_path) as db_conn:
                cursor = db_conn.cursor()
                with open(csv_path, "r") as f:
                    reader = csv.reader(f)
                    next(reader)
                    for row in reader:
                        cursor.execute(
                            "INSERT INTO inventory_pipeline (sku, name, quantity) VALUES (?, ?, ?)",
                            (row[0], row[1].upper(), int(row[2]))
                        )
    return timer.elapsed
```


Explanation: This capstone integrates multiple resource boundaries. Even if the SQL insertions fail midway, database changes are rolled back, temporary directories are unlinked, and performance is correctly logged by the enclosing Timer.